
리눅스 기초 명령어

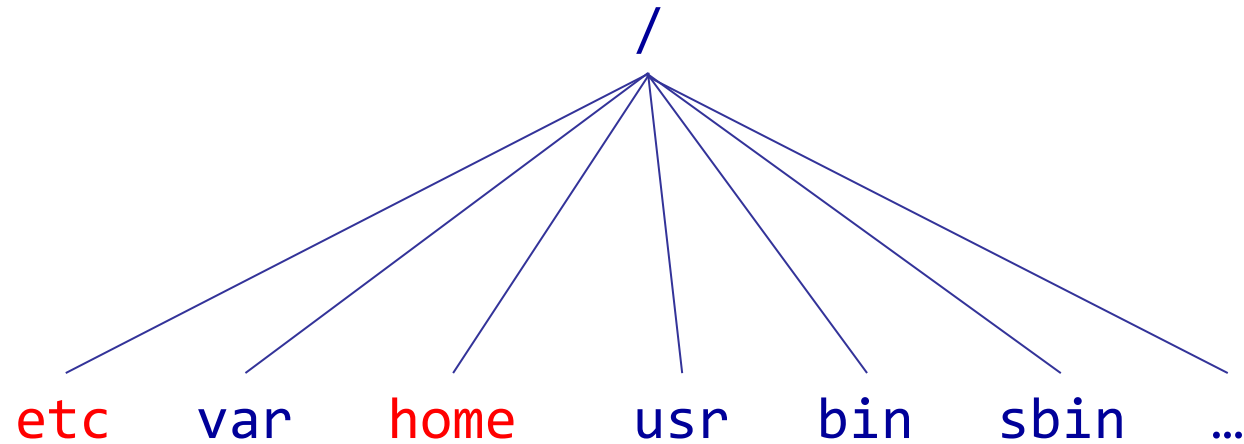
command [-단일문자옵션] [--다중문자옵션]

- 단일문자로 구성된 옵션은 동시에 여러 개를 지정 할 수 있다.

ex) ls -a -l 이런 표현은 다음과 같이 표현 가능하다.
ls -al

- 디렉토리란 파일을 담고 있는 일종의 방이다.
- 디렉토리 및 파일의 표현 방법
 - 절대 경로 표현방법
 - 상대 경로 표현방법
- 절대 경로 표현방법
 - / 디렉토리를 기준으로 표현하는 방법
- 상대경로 표현방법
 - 현재 위치(./) 를 기준으로 표현하는 방법

- 리눅스의 디렉토리는 /(root) 디렉토리를 기점으로 트리 구조를 구성한다.
 - FHS(Filesystem Hierarchy Standard) : FSSTND



➤ 디렉토리의 표현

- / : 최상위 root 디렉토리 (절대경로 표현의 기준)
- ./ : 현재 디렉토리 (상대경로 표현의 기준 : 생략가능)
- ../ : 현디렉토리의 상위 디렉토리
- ~/ : 각 사용자의 홈디렉토리

➤ 디렉토리는 파일의 일종이다

- 이름 : 최대 255자
- 대소문자 구별
- 확장자가 없다.

➤ cd

디렉토리를 이동하는 명령어, 보통 옵션이 없이 사용한다.

```
# cd [이동할 디렉토리명]
```

➤ pwd

현재 위치한 디렉토리를 절대경로 표현법으로 출력한다.

➤ rmdir

지정한 디렉토리를 삭제한다.

디렉토리가 비워져 있는 경우만 삭제 가능하다.

➤ mkdir

지정한 디렉토리를 생성한다.

생성된 디렉토리는 생성한 유저의 소유가 된다.

-p, --parents

: 필요하다면 생성하고자 하는 상위의 디렉토리까지 생성한다.

--version

: 버전 정보를 보여준다.

```
[root@linux ~]# pwd
/root
[root@linux ~]# mkdir a
[root@linux ~]# ls
a  anaconda-ks.cfg
[root@linux ~]# cd a
[root@linux a]# pwd
/root/a
[root@linux a]# cd ..
[root@linux ~]# rmdir a
[root@linux ~]# mkdir ./a/b
mkdir: `./a/b' 디렉토리를 만들 수 없습니다: 그런 파일이나 디렉터리가 없습니다
[root@linux ~]# mkdir -p ./a/b
[root@linux ~]# ls ./a/
b
[root@linux ~]# cd ./a/b
[root@linux b]#
```

➤ tree

디렉토리 구조를 출력한다.

-d : 디렉토리만 출력한다.

```
[root@linux ~]# tree -d /etc/yum
/etc/yum
|-- fssnap.d
|-- pluginconf.d
|-- protected.d
`-- vars
```

- 미 설치 : `dnf install -y tree`

➤ ls

현재 디렉토리의 목록을 출력한다.

- a, --all : 숨겨진 파일의 목록까지 출력
- l : 파일에 대한 정보를 자세히 출력
- R : 하위 디렉토리의 내용까지 출력
- h : 사람이 이해하기 쉽도록 파일의 크기를 출력
- k : kb단위로 파일의 크기출력
- i : 파일의 inode 번호를 같이 출력 (고유번호)

➤ 리다이렉트

표준 입출력 지시자

> : 표준 출력 지시자

command > 파일 : 명령 결과를 파일로 저장

>> : 추가 표준 출력 지시자

command >> 파일 : 명령 결과를 파일에 추가

< : 표준 입력 지시자

command < 파일 : 파일의 내용을 명령의 입력 값으로 사용

➤ cat

cat [옵션] [출력 대상 파일]

옵션

-n : 행 단위 번호

-b : 공백 라인을 제외하고 행 단위 번호

cat a.txt

cat -n a.txt

cat -nb a.txt

➤ cat

파일의 입출력 제어

cat [< 입력파일 > 출력파일]

기본 입력 파일 : 키보드

기본 출력 파일 : CRT

입력 파일의 경우 < 기호 생략 가능

>> : 출력파일이 존재하는 경우 내용추가

cat < a.txt > b.txt

cat < /dev/null > a.txt

cat > b.txt ^c, ^d

cat <<EOF > ./abc.txt

> 내용 입력

> EOF

➤ echo

환경 변수나 입력내용을 출력

```
# echo [환경변수 | 문자열]
```

리다이렉트를 이용하면 파일을 생성할 수도 있다.

ex>

```
# echo $HOME
```

```
# echo '$HOME'
```

➤ env

모든 환경 변수를 출력한다.

```
[root@linux ~]# echo hi daba
hi daba
[root@linux ~]# echo $HOME
/root
[root@linux ~]# echo "$HOME"
/root
[root@linux ~]# echo '$HOME'
$HOME
[root@linux ~]# echo 1 > a.txt
[root@linux ~]# cat a.txt
1
[root@linux ~]# cat > b.txt
I am biro
compound bow
^C
[root@linux ~]# cat b.txt
I am biro
compound bow
```

← ^C, ^D는 모두 종료지만 결과는 다르다.

➤ \$는 변수 연산자

\$PATH, \$CLASS_PATH ...

```
[root@linux ~]# export TEST="AA"  
[root@linux ~]# echo $TEST  
AA
```

➤ \$(command)는 명령의 실행 결과

```
[root@linux ~]# echo 현재 디렉토리는 $(pwd)입니다.  
현재 디렉토리는 /root입니다.  
[root@linux ~]# echo "현재 디렉토리는 $(pwd)입니다."  
현재 디렉토리는 /root입니다.  
[root@linux ~]# echo '현재 디렉토리는 $(pwd)입니다.'  
현재 디렉토리는 $(pwd)입니다.
```

➤ cp

파일이나 디렉토리를 복사한다.

```
# cp [옵션] 원본파일 복사파일명
```

옵션

- a : 원본의 속성유지
- r : 하위 디렉토리나 파일모두 복사
- f : 강제 실행
- p : 파일의 소유와 권한등을 보존한채 복사
- u : 새파일 덮어쓰기 금지

➤ mv

파일이나 디렉토리를 이동한다.

```
# mv [옵션] 원본파일 이동파일명
```

옵션

- f : 강제 실행
- b : 파일이 덮어 쓰여질 경우 백업파일 생성
- t : 이동할 대상의 디렉토리 지정
- d : 디렉토리 삭제
- i : 삭제 전 삭제여부 확인
- v : 백업 파일 생성

➤ rm

파일이나 디렉토리를 삭제한다.

```
# rm [옵션] 대상파일
```

옵션

-f : 강제 실행

-r : 대상 중에 디렉토리가 있는 경우 디렉토리도 삭제

```
[root@linux ~]# cp a.txt c.txt
[root@linux ~]# ls *.txt
a.txt  b.txt  c.txt
[root@linux ~]# ls
a  a.txt  anaconda-ks.cfg  b.txt  c.txt
[root@linux ~]# mv c.txt a/
[root@linux ~]# ls a/
b  c.txt
[root@linux ~]# rm a
rm: cannot remove `a': 디렉터리입니다
[root@linux ~]# rm -rf a
[root@linux ~]# rm *.txt
rm: remove 일반 파일 `a.txt'? y
rm: remove 일반 파일 `b.txt'? y
[root@linux ~]# ls *.txt
ls: cannot access *.txt: 그런 파일이나 디렉터리가 없습니다
```

➤ ln

link 파일 생성 (soft, hard)

ln [옵션] 원본파일 링크파일

옵션

-s : soft link 생성 (디렉토리의 경우 soft link만 가능)

-f : 대상이 존재하는 경우 지우고 link파일 생성

➤ 심볼릭(소프트) 링크(symbolic link)와 하드 링크(hard link)

- 심볼릭 링크 : 원본 파일의 위치정보를 가지는 단축 아이콘으로 특정 파일을 가리키고 있는 파일이다. 다른 파일시스템에서도 링크가 가능하다. 별도의 inode를 가지며 링크 파일이 가지고 있는 용량만큼 디스크의 공간을 차지하며 원본의 링크 카운트가 증가하지 않는다.
- 하드 링크 : 원본 파일과 동일한 inode를 가진다. 그렇기 때문에 원본이었던 파일이 삭제 되더라도 동일 inode를 갖고 있는 링크 파일은 여전히 사용 가능하다. 서로 다른 파일시스템에서는 생성이 불가능하며 링크 카운트가 증가한다.

```
[root@linux ~]# echo aa > 1.txt
[root@linux ~]# ln 1.txt 2.txt
[root@linux ~]# ls -li *.txt
33585638 -rw-r--r-- 2 root root 3  9월  7 14:33 1.txt
33585638 -rw-r--r-- 2 root root 3  9월  7 14:33 2.txt

[root@linux ~]# ln -s 1.txt 3.txt
[root@linux ~]# ls -li *.txt
33585638 -rw-r--r-- 2 root root 3  9월  7 14:33 1.txt
33585638 -rw-r--r-- 2 root root 3  9월  7 14:33 2.txt
33585639 lrwxrwxrwx 1 root root 5  9월  7 14:34 3.txt -> 1.txt

[root@linux ~]# rm 1.txt
rm: remove 일반 파일 `1.txt'? y
[root@linux ~]# ls -li *.txt
33585638 -rw-r--r-- 1 root root 3  9월  7 14:33 2.txt
33585639 lrwxrwxrwx 1 root root 5  9월  7 14:34 3.txt -> 1.txt
```

➤ more(less)

화면 출력을 화면 단위로 제어

출력되는 내용이 많아서 화면을 넘어가는 경우 사용

```
# more 파일
```

➤ |

앞 명령의 결과를 뒤 명령의 입력 값으로 이용

```
# ls -al | more
```

➤ grep

필터링

출력 문자열중에 원하는 단어나 글자가 들어있는 라인만 출력하고자 할때 이용

```
command | grep [필터링 문자열]
```

```
ls -al | grep conf
```

```
[root@linux ~]# cat -n /etc/passwd | grep ftp
    12  ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
```

```
[root@linux ~]# env | grep HOME
HOME=/root
```

➤ head, tail

파일의 일부분을 출력. 각각 앞 뒤에서 10 라인 씩 출력

옵션

- n ## : 지정한 라인만큼 출력
- c ## : 지정한 byte 만큼 출력
- f : tail에서만 사용, 실시간 추가되는 내용도 출력
주로 log 정보 확인 시 사용한다.

➤ truncate

파일의 크기를 임의로 변경한다.

파일의 내용을 삭제할 때 주로 쓰인다.

옵션

- s N : 파일 크기를 N 바이트로 설정
 - . 단위 지정 가능 : K(KB), M(MB), G(GB)
 - . 크기 증감 가능 : + / -
- c : 파일이 존재할 경우만 실행한다.
- r file : 제공한 파일의 크기를 기준으로 설정한다.

```
[root@linux ~]# ls /etc/ > list.txt
```

```
[root@linux ~]# head -n 5 list.txt
```

```
DIR_COLORS  
DIR_COLORS.256color  
DIR_COLORS.lightbgcolor  
GREP_COLORS  
GeoIP.conf
```

```
[root@linux ~]# tail -f -n 5 /var/log/messages
```

```
Sep  7 16:31:58 linux systemd: Mounted Arbitrary Executable File Formats File System.  
Sep  7 16:40:01 linux systemd: Started Session 13 of user root.  
Sep  7 16:50:01 linux systemd: Started Session 14 of user root.  
Sep  7 17:00:01 linux systemd: Started Session 15 of user root.  
Sep  7 17:01:01 linux systemd: Started Session 16 of user root.
```

```
[root@linux ~]# ls -alh list.txt
-rw-r--r-- 1 root root 2.0K  6월 21 17:42 list.txt

[root@linux ~]# truncate -s +1M list.txt
[root@linux ~]# ls -alh list.txt
-rw-r--r-- 1 root root 1.1M  6월 21 17:44 list.txt

[root@linux ~]# truncate -s 10 list.txt
[root@linux ~]# ls -alh list.txt
-rw-r--r-- 1 root root 10   6월 21 17:44 list.txt

[root@linux ~]# truncate -s 0 list.txt
[root@linux ~]# ls -alh list.txt
-rw-r--r-- 1 root root 0    6월 21 17:45 list.txt
```

- 설정 파일이나 로그 파일의 내용을 삭제하는 경우 데몬의 파일 핸들이 유지됨으로 truncate 명령을 주로 사용한다.

➤ find

파일시스템에서 조건에 만족하는 파일을 검색한다.

```
# find  검색경로  검색조건1  검색조건2  ...
```

검색조건

-name	: 파일명으로 검색 (와일드카드문자 사용)
-perm	: 특정 퍼미션을 가진 파일을 검색
-empty	: 크기가 0인 파일 검색

검색조건

-user	지정한 사용자가 소유한 파일
-group	지정한 그룹이 소유한 파일
-nouser	소유자가 없는 파일
-nogroup	그룹 소유자가 없는 파일
-type	검색하고자 하는 파일의 종류 지정 b(block), c(char), d(dir), l(slink), f(regular)
-mount	마운트된 파일 시스템만 검색
-empty	빈 파일 검색
-mtime, -atime, -ctime	변경된 파일 (-숫자)

mtime:내용 변경, ctime:정보 변경, atime:접근날짜

```
[root@linux ~]# find /var/ -name *.conf
/var/lib/NetworkManager/NetworkManager-intern.conf

[root@linux ~]# find /etc/ /var/ -name ???*.conf
/etc/lvm/lvm.conf
/etc/lsm/pluginconf.d/sim.conf
/etc/yum.conf
/etc/nfs.conf
/etc/sos.conf

[root@linux ~]# find /dev/ -name sd* -type b
/dev/sda2
/dev/sda1
/dev/sda
```

➤ find 명령 에러

find: paths must precede expression:.....

-name 옵션 사용시 문자열을 인식하지 못하는 에러이다.
이런 경우 문자열에 ' '를 추가한다.

find / -name *.txt ➡ find / -name '*.txt'

➤ tar

여러 개의 파일을 하나의 파일로 압축한다. 단 용량은 줄어들지 않는다. 용량을 줄이기 위해서는 gzip을 이용하거나 특별한 옵션을 이용한다.

```
# tar [옵션] [압축파일명] [대상]
```

- c : 압축, -x : 풀기, -v : 작업내용을 출력
- z : gzip를 이용 압축(압축해제), -j : bzip2 이용
- f : 파일명을 지정, -p : 퍼미션 유지

```
[root@linux ~]# mkdir -p ./aa/a/
[root@linux ~]# echo 1 > ./aa/a.txt
[root@linux ~]# echo 2 > ./aa/a/a.txt
[root@linux ~]# tar cvfz aa.tar.gz aa
aa/
aa/a/
aa/a/a.txt
aa/a.txt
[root@linux ~]# rm -rf aa
[root@linux ~]# ls
aa.tar.gz  anaconda-ks.cfg
[root@linux ~]# tar xvfz aa.tar.gz
aa/
aa/a/
aa/a/a.txt
aa/a.txt
[root@linux ~]# ls -R aa
```

➤ tar 를 이용한 증분 백업

tar [옵션] [압축파일명] -g [백업정보 파일] [대상]

Ex)

```
# tar cvfz home_full.gz -g /backup/home_backup /home
```

```
# tar cvfz home_11.gz -g /backup/home_backup /home
```

```
# tar cvfz home_12.gz -g /backup/home_backup /home
```

-> 복구는 반드시 압축순으로 해제한다.

- Os 레벨에서의 완전한 증분 백업은 불가능하다.

: 파일 단위 백업이므로.. → 그럼 파일 단위가 아니면 ??

```
[root@linux ~]# rm -rf *.txt
[root@linux ~]# rm -rf aa*

[root@linux ~]# mkdir backup
[root@linux ~]# mkdir imsy
[root@linux ~]# echo 1 > ./imsy/a.txt
[root@linux ~]# echo 2 > ./imsy/b.txt
[root@linux ~]# tar cvfz ./backup/full.tar.gz -g ./backup/imsy ./imsy/
tar: ./imsy: Directory is new
./imsy/
./imsy/a.txt
./imsy/b.txt
[root@linux ~]# echo 3 >> ./imsy/b.txt
[root@linux ~]# tar cvfz ./backup/cum1.tar.gz -g ./backup/imsy ./imsy/
./imsy/
./imsy/b.txt
```

➤ gzip

- # gzip a.txt : a.txt.gz로 압축한다.
- # gzip -d a.txt.gz : 파일의 압축을 해제한다.

➤ bzip2, compress, cpio

```
# bzip2, compress : 압축
# bunzip2, uncompress : 압축 해제

# cpio -idmv < [압축파일명] : 압축해제
```

➤ stat

파일의 시간 정보를 출력한다.

stat 파일

```
[root@linux ~]# stat a.txt
  File: `a.txt'
  Size: 2                Blocks: 8          IO Block: 4096   일반 파일
Device: fd00h/64768d    Inode: 51342985   Links: 1
Access: (0644/-rw-r--r--)  Uid: (   0/   root)   Gid: (   0/   root)
Access: 2022-09-07 15:55:31.361025761 +0900
Modify: 2022-09-07 15:54:37.000000000 +0900
Change: 2022-09-07 16:04:28.905000893 +0900
 Birth: -
```

➤ 리눅스 파일의 타임 스탬프

- `atime`
 - 파일에 마지막으로 접근한 시간
 - 시스템이나 응용프로그램이 마지막으로 파일에 접근한 시간
- `ctime`
 - 파일의 퍼미션 등 속성이 변경된 시간.
- `mtime`
 - 파일의 내용이 마지막으로 수정된 시간

```
[root@linux ~]# stat a.txt
```

```
.....
```

```
Access: 2022-09-07 15:55:31.361025761 +0900
```

```
Modify: 2022-09-07 15:54:37.000000000 +0900
```

```
Change: 2022-09-07 16:04:28.905000893 +0900
```

```
Birth: -
```

```
[root@linux ~]# chmod 755 a.txt
```

```
[root@linux ~]# stat a.txt
```

```
.....
```

```
Access: 2022-09-07 15:55:31.361025761 +0900
```

```
Modify: 2022-09-07 15:54:37.000000000 +0900
```

```
Change: 2022-09-07 16:06:20.851001915 +0900
```

```
Birth: -
```

➤ touch

파일의 시간 정보를 갱신한다. 파일이 없으면 빈 파일을 생성한다.

touch [옵션] 파일

옵션

- t [YYYYMMDDhhmi.ss] : 임의의 시간으로 변경, a,mtime 변경
- a : 최근 사용 시간 변경, a,ctime 변경
- m : 파일의 최근 수정 시간 변경, m,ctime 변경
- r [file] : 지정한 파일과 동일한 시간으로 변경
- c : 파일을 생성하지 않는다.(존재하지 않을 때)

```
[root@linux ~]# touch test.txt
[root@linux ~]# ls -al test.txt
-rw-r--r-- 1 root root 0  9월  7 16:15 test.txt
```

```
[root@linux ~]# touch -t 199901010001.30 test.txt
[root@linux ~]# ls -al test.txt
-rw-r--r-- 1 root root 0  1월  1 1999 test.txt
```

```
[root@linux ~]# stat test.txt
```

File: `test.txt'

Size: 0	Blocks: 0	IO Block: 4096	일반 빈 파일
Device: fd00h/64768d	Inode: 33585638	Links: 1	
Access: (0644/-rw-r--r--)	Uid: (0/ root)	Gid: (0/ root)	
Access: 1999-01-01 00:01:30.000000000	+0900		
Modify: 1999-01-01 00:01:30.000000000	+0900		
Change: 2022-09-07 16:16:14.435007333	+0900		
Birth: -			